

Claude Code 系统核心提示词防泄露机制深度分析

> 对 Claude Code CLI 项目中系统提示词 (System Prompt) 的保护措施的全面剖析

> 调研日期: 2026-07-08

目录

1. 整体架构概览
2. 系统提示词的存储与加载
3. 编译时防泄露——Feature Flag 死代码消除
4. 运行时防泄露——Undercover Mode
5. 子智能体提示词隔离
6. 调试接口保护——Dump Prompts 系统
7. 提示词缓存与边界控制
8. 总结：多层防泄露体系
9. 关键文件索引

1. 整体架构概览

为什么要保护系统提示词？

Claude Code 的系统提示词 (system prompt) 是整个 CLI 的行为说明书——它告诉 AI 模型：

- 它的身份是什么 ("You are Claude Code...")
- 它应该如何工作 (工具使用规则、代码风格、安全要求)
- 它应该如何沟通 (语气、效率、输出格式)
- 特有的安全约束 (Cyber-risk 指令、动作审查指南)

如果这些提示词泄露出去，会导致：

1. 安全边界暴露——攻击者可以针对性地构造 prompt injection 绕过规则
2. 知识产权泄露——Anthropic 内部的 prompt engineering 策略被公开
3. 未发布功能曝光——内部模型代号、实验性功能在开源仓库中被看到

防泄露体系总览

[illegible]

[illegible]

2. 系统提示词的存储与加载

为什么需要动态构建?

系统提示词并非一个写死的静态文本文件。它需要根据当前环境动态生成：

```
// src/constants/prompts.ts:444-577
export async function getSystemPrompt(
  tools: Tools,
  model: string,
  additionalWorkingDirectories?: string[],
  mcpClients?: MCPServerConnection[],
): Promise
```

为什么要动态构建？因为每个用户的会话不同：

- 不同的工具集 (MCP 工具、feature flag 控制的工具)
- 不同的环境 (操作系统、工作目录、git 状态)
- 不同的设置 (语言偏好、输出风格、记忆内容)
- 不同的 MCP 服务器 (每个服务器有自己的 instructions)

三阶段构建流程

```

1: getSimpleIntroSection() → getSimpleSystemSection() → getSimpleDoingTasksSection() → getActionsSection() → getUsingYourToolsSection() → getSimpleToneAndStyleSection() → getOutputEfficiencySection()
SYSTEM_PROMPT_DYNAMIC_BOUNDARY
2: Section memoized
session_guidance → memory → env_info_simple → language → output_style → mcp_instructions → MCP scratchpad → frc → summarize_tool_results
3: API
buildSystemPromptBlocks()
splitSysPromptPrefix() → Anthropic API

```

完整提示词内容

以下是 `getSimpleIntroSection()` 生成的身份与安全指令：

以下是 `getSimpleSystemSection()` 生成的系统规则:

以下是 `getSimpleDoingTasksSection()` 生成的任务执行指南（简化版）：

3. 编译时防泄露——Feature Flag 死代码消除

这是最底层、最彻底的防泄露措施。Claude Code 使用 Bun 的 `bun:bundle` 编译时 feature flag 系统，配合 `process.env.USER_TYPE` 构建常量，在编译阶段就将敏感代码从外部构建中完全删除。

process.env.USER TYPE 常量折叠

```
// ■ 621-628: Undercover ■■■■■■■■■■
let modelDescription = ''
```

```

if (process.env.USER_TYPE === 'ant' && isUndercover()) {
// suppress - ■■■■ USER_TYPE !== 'ant'■■■■■
} else {
const marketingName = getMarketingNameForModel(modelId)
modelDescription = marketingName
? `You are powered by the model named ${marketingName}. The exact model ID is ${modelId}.`
: `You are powered by the model ${modelId}.`
}

// ■ 529-537: ant-only ■■■■■■
...(process.env.USER_TYPE === 'ant'
? [systemPromptSection('numeric_length_anchors', () =>
'Length limits: keep text between tool calls to ≤25 words...')
])
: []),

// ■ 403-414: ant-only ■■■■■■■■■■
function getOutputEfficiencySection(): string {
if (process.env.USER_TYPE === 'ant') {
return `# Communicating with the user\nWhen sending user-facing text...` // ■■■
}
return `# Output efficiency\nIMPORTANT: Go straight to the point...` // ■■
}

```

feature() 宏控制

```

// src/entrypoints/cli.tsx:50-69
// --dump-system-prompt ■■■■ DUMP_SYSTEM_PROMPT feature flag ■■■■
if (feature('DUMP_SYSTEM_PROMPT') && args[0] === '--dump-system-prompt') {
const prompt = await getSystemPrompt([], model)
console.log(prompt.join('\n'))
return
}

```

feature('DUMP_SYSTEM_PROMPT') 是编译时宏——在外部构建中，它被求值为 false，整个 if 块被消除。外部用户即使尝试 --dump-system-prompt 也无法触发。

保护的代码范围

以下 ant-only 代码块全部通过 USER_TYPE === 'ant' 或 feature() 保护：

代码区域 | 保护机制 | 敏感内容

`computeEnvInfo()` 中的模型名称 | `USER\_TYPE` | 内部模型代号和版本号

`getSimpleDoingTasksSection()` 中的注释规则 | `USER\_TYPE` | 特定模型的行为调优

`getOutputEfficiencySection()` 详细版 | `USER\_TYPE` | 内部沟通风格指南

`numeric\_length\_anchors` section | `USER\_TYPE` | ant-only token 优化

`--dump-system-prompt` 入口 | `feature('DUMP\_SYSTEM\_PROMPT')` | 完整系统提示词导出

`dumpPrompts.ts` 全部代码 | `USER\_TYPE` | 完整 API 请求/响应

`undercover.ts` 全部代码 | `USER\_TYPE` | 内部仓库检测逻辑

虚假声明缓解指令 | `USER\_TYPE` | 特定模型行为调优

用户反馈路由 | `USER\_TYPE` | 内部 Slack 频道和流程

4. 运行时防泄露——Undercover Mode

为什么需要 Undercover Mode?

编译时 DCE 保护的是外部构建本身。但 Anthropic 内部员工使用内部构建（包含所有 ant-only 代码）时，也可能在公共/开源仓库中工作。Undercover Mode 就是为这个场景设计的——防止内部员工在公开仓库中不小心泄露内部信息。

自动激活机制

```
// src/utils/undercover.ts:28-37
export function isUndercover(): boolean {
  if (process.env.USER_TYPE === 'ant') {
    if (isEnvTruthy(process.env.CLAUDE_CODE_UNDERCOVER)) return true // ■■■■
    return getRepoClassCached() !== 'internal' // ■■■■
  }
  return false // ■■■■■■■■ false
}
```

自动检测逻辑：

- 检查当前仓库的 git remote 是否匹配 INTERNAL_MODEL_REPOS 白名单
- 如果匹配 → repoClass === 'internal' → Undercover 关闭
- 如果不匹配 → repoClass === 'external' | 'none' | null → Undercover 开启
- 不可手动关闭——安全默认是开启的

```
■■■■■■■■■■
■
▼
getRepoClassCached()
■
■■■■■■■■■■■■■■■■■■■■
■ ■
▼ ▼
internal ■■■external/none/null■
■ ■
▼ ▼
Undercover Undercover
■■ ■■■■■■■■
```

Undercover 做了什么？

1. 隐藏模型身份——系统提示词中不包含模型名称和 ID：

```
// prompts.ts:620-628
let modelDescription = ''
if (process.env.USER_TYPE === 'ant' && isUndercover()) {
  // suppress - ■■■■■■
} else {
  const marketingName = getMarketingNameForModel(modelId)
  modelDescription = marketingName
  ? `You are powered by the model named ${marketingName}. The exact model ID is ${modelId}.`
  : `You are powered by the model ${modelId}.`
}
```

2. 在 commit/PR 提示词中注入防泄露指令：

```
// undercover.ts:39-72
export function getUndercoverInstructions(): string {
  return `## UNDERCOVER MODE - CRITICAL
You are operating UNDERCOVER in a PUBLIC/OPEN-SOURCE repository...

NEVER include in commit messages or PR descriptions:
- Internal model codenames (animal names like Capybara, Tengu, etc.)
- Unreleased model version numbers (e.g., opus-4-7, sonnet-4-8)
- Internal repo or project names
- Internal tooling, Slack channels, or short links
- The phrase "Claude Code" or any mention that you are an AI
- Any hint of what model or version you are
- Co-Authored-By lines or any other attribution`
}
```

安全哲学

> "Safe default is ON — Claude may push to public remotes from a CWD that isn't itself a git checkout (e.g. /tmp crash repro). There is NO force-OFF."

这是一个精心设计的失败安全（fail-safe）策略：任何时候不确定是否在内部仓库，就保持 Undercover。宁可误判也不泄露。

5. 子智能体提示词隔离

为什么子智能体需要隔离？

当主 agent 使用 AgentTool 创建子智能体时，子智能体执行的是局部任务（搜索代码、运行测试、审查代码）。子智能体不应该知道：

- 主 agent 的完整身份和规则
- Cyber-risk 指令
- 输出风格和效率要求
- 安全和动作审查指南

最小提示词策略

```
// src/tools/AgentTool/runAgent.ts:906-932
async function getAgentSystemPrompt(
  agentDefinition: AgentDefinition,
  toolUseContext: Pick,
  resolvedAgentModel: string,
  additionalWorkingDirectories: string[],
  resolvedTools: readonly Tool[],
): Promise {
  try {
    const agentPrompt = agentDefinition.getSystemPrompt({ toolUseContext })
    const prompts = [agentPrompt] // ← 最小提示词
    return await enhanceSystemPromptWithEnvDetails(
      prompts, resolvedAgentModel, additionalWorkingDirectories, enabledToolNames,
    )
  } catch (_error) {
    return enhanceSystemPromptWithEnvDetails(
      [DEFAULT_AGENT_PROMPT], // ← 最小提示词
      resolvedAgentModel, additionalWorkingDirectories, enabledToolNames,
    )
  }
}
```

子智能体的系统提示词只有两个部分：

```

Agent DEFAULT_AGENT_PROMPT
  "You are an agent for Claude Code... Complete the task fully-don't gold-plate, but don't leave it half-done. When you complete the task, respond with a concise report..."
  enhanceSystemPromptWithEnvDetails
    - 
    - 
    - git
```

主 Agent vs 子 Agent 提示词对比

维度	主 Agent (~5000 tokens)	子 Agent (~500 tokens)
身份声明	"You are Claude Code..."	"You are an agent for Claude Code..."
Cyber-risk 指令	有	无
系统规则	完整 (~8 条)	无
任务执行指南	完整 (代码风格、安全等)	无
动作审查指南	完整 (可逆性、破坏性操作)	无

工具使用规则 | 完整（优先工具而非 bash） | 无
语气和风格 | 完整（无表情符号等） | 无
输出效率规则 | 完整 | 无
环境信息 | 完整（含模型名称） | 简化版
会话记忆 | 有 | 无
MCP 指令 | 有 | 仅 agent 指定

Forked Agent 的特殊处理

当 AgentTool 不带 subagent_type 调用时（fork 模式），子进程继承父进程的上下文以共享缓存。此时系统提示词通过 CacheSafeParams 传递：

```
// src/utils/forkedAgent.ts
export type CacheSafeParams = {
  systemPrompt: SystemPrompt // ████████████████████
  userContext: { [k: string]: string }
  systemContext: { [k: string]: string }
  toolUseContext: ToolUseContext
  forkContextMessages: Message[]
}
```

但这不会导致泄露——fork 进程的执行结果通过 SendMessage 返回，其工具输出不会流入主进程的上下文窗口。整个设计确保“fork 看到的 prompt 与主进程相同”仅是缓存优化的副作用，不是安全漏洞。

验证提示隔离

回顾第 4 章中 TodoWriteTool 的验证提示（verification nudge）——条件③明确要求 !context.agentId：

```
// TodoWriteTool.ts:77-86
if (
  feature('VERIFICATION_AGENT') && // ① Feature Flag
  getFeatureValue_CACHED_MAY_BE_STALE(...) && // ② ████████
  !context.agentId && // ③ ██████████ agent
  allDone && // ④ ██████
  todos.length >= 3 && // ⑤ █ 3 █
  !todos.some(t => /verif/i.test(t.content)) // ⑥ ██████
) { verificationNudgeNeeded = true }
```

条件③确保子智能体关闭 todo 列表时不会触发验证提示——因为子智能体不应该知道验证 agent 的存在。

6. 调试接口保护——Dump Prompts 系统

功能概述

dumpPrompts.ts 是一个调试工具，用于捕获所有 API 请求和响应的完整内容（包括系统提示词、工具定义、用户消息）。它在 Anthropic 内部用于：

- 调试 prompt 相关问题
- 为 /issue 命令提供上下文
- 分析模型行为

保护措施

措施 1: USER_TYPE 门控

```
// src/services/api/dumpPrompts.ts:48-57
export function addApiRequestToCache(requestData: unknown): void {
  if (process.env.USER_TYPE !== 'ant') return // ← ██████████
  cachedApiRequests.push({...})
}
```

措施 2: 有限缓存

```
MAX_CACHED_REQUESTS = 5 // 5
```

只保留最近的 5 条请求，用于 /issue 命令的诊断数据。不长期存储。

措施 3: 文件路径隔离

```
// dumpPrompts.ts:59-65
export function getDumpPromptsPath(agentIdOrSessionId?: string): string {
  return join(getClaudeConfigHomeDir(), 'dump-prompts', `${agentIdOrSessionId}.jsonl`)
}
```

输出到 ~/.claude/dump-prompts/ 目录，按 session 隔离，不在工作目录中产生文件。

措施 4: 写入为 fire-and-forget

```
// dumpPrompts.ts:67-72
function appendToFile(filePath: string, entries: string[]): void {
  if (entries.length === 0) return
  fs.mkdir(dirname(filePath), { recursive: true })
  .then(() => fs.appendFile(filePath, entries.join('\n') + '\n'))
  .catch(() => {}) // ←
}
```

写入失败不会影响正常使用。这是调试工具，不是关键路径。

完整请求/响应捕获流

```
query.ts  fetch
▼
dumpPromptsFetch  ant
  (32  Anthropic API
  body.model,
  body.system,
  body.messages,
  response.*)
  ~/.claude/ Anthropic API
  dump-prompts/ ( )
  .jsonl
```

7. 提示词缓存与边界控制

为什么需要缓存控制？

系统提示词很大 (~5000 tokens)，每次重新发送会浪费大量 tokens。API 级 prompt caching 允许缓存静态部分，只重新发送动态变化的部分。但这也带来了安全考虑——需要确保缓存范围不会导致跨会话泄露。

SYSTEM_PROMPT_DYNAMIC_BOUNDARY

```
// src/constants/prompts.ts:114-115
export const SYSTEM_PROMPT_DYNAMIC_BOUNDARY = '__SYSTEM_PROMPT_DYNAMIC_BOUNDARY__'
```

这是一个标记字符串，放在静态内容和动态内容之间：

```

[0]  + Cyber-risk ← cache_control: { type: 'global' }
[1]  ← cache_control: { type: 'global' }
[2]  ← cache_control: { type: 'global' }
```



```
[3] ██████ ← cache_control: { type: 'global' }
[4] ██████ ← cache_control: { type: 'global' }
[5] █████ ← cache_control: { type: 'global' }
[6] █████ ← cache_control: { type: 'global' }
[7] __SYSTEM_PROMPT_DYNAMIC_BOUNDARY__ ← █████
[8] █████ ← █████
[9] █████ ← █████
[10] █████ ← █████
... ← █████
```

splitSysPromptPrefix 缓存范围分割

```
// src/utils/api.ts:321-440
function splitSysPromptPrefix(system, fingerprint, isGlobalCacheMode, mcpTools):
// ███ CLI_SYSPROMPT_PREFIXES ██████
// ███ SYSTEM_PROMPT_DYNAMIC_BOUNDARY
// █████ 4 █████
// █0: attribution header█n/a scope█
// █1: CLI prefix█n/a scope█
// █2: █████global scope█
// █3: █████n/a scope█
// █ MCP █████ org-level scope
```

三种身份前缀

```
// src/constants/system.ts:10-28
const DEFAULT_PREFIX = `You are Claude Code, Anthropic's official CLI for Claude.`
const AGENT_SDK_CLAUDE_CODE_PRESET_PREFIX = `You are Claude Code, Anthropic's official CLI for Claude,
running within the Claude Agent SDK.`
const AGENT_SDK_PREFIX = `You are a Claude agent, built on Anthropic's Claude Agent SDK.`
```

这些前缀用于 splitSysPromptPrefix 通过内容匹配（而非位置）来识别身份块，确保即使 prompt 结构变化，缓存仍能正确工作。

Memoized Section 系统

```
// src/constants/systemPromptSections.ts:1-69
export function systemPromptSection(name: string, compute: ComputeFn): SystemPromptSection
export function DANGEROUS_uncachedSystemPromptSection(name: string, compute: ComputeFn, _reason:
string): SystemPromptSection
export async function resolveSystemPromptSections(sections: SystemPromptSection[]): Promise<(string |
null)[]>
```

- systemPromptSection — 创建 memoized section。计算一次，缓存到 /clear 或 /compact
- DANGEROUS_uncachedSystemPromptSection — 创建每次重新计算的 section。名称中的 DANGEROUS_ 是显式的警告，因为每次重新计算都会破坏提示词缓存。需要传入 _reason 参数解释为什么必须打破缓存
- resolveSystemPromptSections — 批量解析所有 sections，优先使用缓存值

当前使用的 sections:

Section 名称 | 是否缓存 | 推断原因

`session\_guidance` | 是 | 会话开始后不变

`memory` | 是 | 会话开始后不变

`ant\_model\_override` | 是 | 不变

`env\_info\_simple` | 是 | 会话开始后不变

`language` | 是 | 不变

`output\_style` | 是 | 不变

`mcp\_instructions` | **否** | MCP 服务器随时可能连接/断开

`scratchpad` | 是 | 不变

`frc` | 是 | 不变

`summarize\_tool\_results` | 是 | 不变

`numeric\_length\_anchors` | 是 | ant-only, 不变

`token\_budget` | 是 | 不变

8. 总结：多层防泄露体系

防御层总结

层级 | 机制 | 文件 | 保护范围 | 绕过难度

****L1: 编译时 DCE**** | ``feature()`` + ``USER_TYPE`` 常量折叠 | 全项目 | 所有 ant-only 功能 |

****极高**** (需要修改构建过程)

****L2: Undercover Mode**** | 自动仓库检测 + 信息隐藏 | `undercover.ts` | 公开仓库中的 ant 构建用户 | 高 (不可手动关闭)

****L3: 子 Agent 隔离**** | 最小提示词策略 | `runAgent.ts` | 所有子智能体 | 高 (独立系统提示词)

****L4: 调试接口保护**** | USER_TYPE 门控 + 有限缓存 | `dumpPrompts.ts` | API 调试数据 | 极高 (编译时消除)

****L5: 缓存边界控制**** | 动态边界标记 + 范围分割 | ``api.ts`` | 提示词缓存范围 | 中 (不影响泄露, 仅防缓存泄露)

数据流安全图

[illegible]

设计哲学总结

1. 编译时 > 运行时: 最敏感的功能在编译阶段就被消除, 不依赖运行时行为
2. Fail-safe 默认: 不确定就开启保护 (Undercover Mode 不可关闭)
3. 最小权限: 子智能体只获得完成任务所需的最小提示词
4. 防御深度: 即使一层被突破, 还有其他层在保护
5. 可审计: 所有 ant-only 代码通过 `USER TYPE === 'ant'` 显式标记, 清晰可审计

9. 关键文件索引

文件 | 用途 | 重要性

```
`src/constants/prompts.ts` | 系统提示词主文件 (~770 行), 所有 prompt section 定义 | □□□□□
```

`src/constants/systemPromptSections.ts` | Section 注册表、memoization、缓存控制 | □□□□
`src/constants/system.ts` | CLI_SYSPROMPT_PREFIXES (三种身份前缀) | □□□
`src/utls/undercover.ts` | Undercover Mode 自动检测和指令生成 | □□□□□
`src/utls/commitAttribution.ts` | INTERNAL_MODEL_REPOS 白名单、getRepoClassCached() | □□□□
`src/services/api/dumpPrompts.ts` | API 请求/响应捕获调试工具 (ant-only) | □□□
`src/services/api/claude.ts` | buildSystemPromptBlocks、API 调用中的系统提示组装 | □□□□
`src/utls/api.ts` | splitSysPromptPrefix (缓存范围分割)、appendSystemContext | □□□□
`src/utls/systemPromptType.ts` | SystemPrompt branded type 定义 | □□
`src/utls/systemPrompt.ts` | buildEffectiveSystemPrompt (优先级决策) | □□□□
`src/tools/AgentTool/runAgent.ts` | 子 agent 系统提示词构建、隔离 | □□□□□
`src/utls/forkedAgent.ts` | Forked agent 的 CacheSafeParams 传递 | □□□
`src/utls/attachments.ts` | critical_system_reminder attachment 注入 | □□
`src/query.ts` | query() 函数中的系统提示词上下文附加 | □□□
`src/entrypoints/cli.tsx` | --dump-system-prompt 入口 (feature flag 控制) | □□
`src/utls/envUtils.ts` | getClaudeConfigHomeDir()、环境工具 | □